

Fashion Image Caption Generation

(with the use of CNN, Transformer, and Metadata)

Project Report

Presented by: Vishal Fulsundar

Instructor: Prof. Amir Jafari, PhD

Institution: George Washington University

Contents

1	Introduction	5
2	Dataset and Preprocessing	6
2.1	Data Source	6
2.2	Data Cleaning and Integration	6
2.3	Train–Validation–Test Split	6
2.4	Vocabulary Construction	6
3	Model Architectures	7
3.1	Baseline Model: Image-Only CNN–Transformer	8
3.1.1	Input Representation	8
3.1.2	ResNet-50 Encoder	8
3.1.3	Transformer Decoder	9
3.1.4	Training	9
3.2	Model-1: Image + Metadata Fusion Architecture	10
3.2.1	Input Representation	10
3.2.2	MobileNetV2 Encoder	10
3.2.3	Fusion Layer	10
3.2.4	Decoder and Training	11
4	Model Evaluation	12
4.1	Evaluation Protocol	12
4.2	Evaluation Metrics	12
4.2.1	BLEU- n	12
4.2.2	ROUGE-L	12
4.3	Quantitative Results for Model-1	13
4.4	Qualitative Case Study: Nike “Splatter” Example	13
4.5	Summary of Model-1 Evaluation	14
5	Explainability	15
5.1	Gradient-Based Token Importance for Baseline and Model-1	16
5.2	Token Similarity Explainability for Qwen2-VL	18
5.3	Why These Explainability Methods Are Meaningful	20
5.4	Summary	21
6	Results and Discussion	22
6.1	Strengths	22
6.2	Limitations	22

6.3	Future Work	22
7	Conclusion	23
8	References	24

Abstract

E-commerce platforms maintain massive fashion catalogs where product descriptions are crucial for search relevance, user experience, and accessibility. Manual caption writing is expensive, slow, and often inconsistent, leading to generic and uninformative descriptions such as “Blue printed shirt for men” instead of “Men’s slim-fit blue printed cotton shirt, casual wear”. In this project, we build an automated fashion image captioning pipeline, beginning with a baseline encoder–decoder model using ResNet-50 and a Transformer decoder. We then extend this into a multimodal architecture (Model-1) that integrates image features with structured metadata via learnable embeddings and a fusion layer. We benchmark our models qualitatively against Qwen2-VL and implement an explainability module using gradient-based token importance. Our results show that metadata fusion significantly improves semantic accuracy, especially for garment type and gender, producing captions close to catalog-quality descriptions. A Streamlit interface demonstrates the models interactively.

1 Introduction

E-commerce platforms rely heavily on accurate and descriptive product captions for search optimization, user understanding, and accessibility. Manual caption writing is labor-intensive and inconsistent, especially at the scale of hundreds of thousands of products. This motivates the use of deep learning models to automate caption generation from product images and metadata.

Fashion products naturally contain multimodal information:

- Visual features: color, silhouette, texture, patterns;
- Structured metadata: gender, category, brand, season, article type.

Problem Statement: Given a product image and its metadata, generate an accurate, catalog-style caption and study how metadata fusion and large VLMs influence caption quality and interpretability.

Objectives:

1. Build an end-to-end data ingestion pipeline.
2. Train a baseline CNN + Transformer captioning model.
3. Extend to a metadata-fusion multimodal model (Model-1).
4. Benchmark against Qwen2-VL.
5. Add explainability using gradient-based token importance.
6. Deploy a Streamlit demo application.

2 Dataset and Preprocessing

2.1 Data Source

We use the publicly available Myntra Fashion Product Dataset from Kaggle, which consists of:

- Product images in JPEG format,
- Metadata table `styles.csv` with attributes: `id`, `gender`, `masterCategory`, `subCategory`, `articleType`, `baseColour`, `brand`, `season`, `year`, `usage`, and `productDisplayName`.

2.2 Data Cleaning and Integration

Preprocessing includes:

1. Standardizing column names and dropping incomplete entries.
2. Linking each product ID to `images/<id>.jpg`.
3. Removing corrupted or unreadable images via PIL.
4. Creating a cleaned caption from `productDisplayName`, e.g.,

“Women Red Sleeveless Fit & Flare Dress” → “women red sleeveless dress”.

2.3 Train–Validation–Test Split

We split the dataset into:

- 70–80% training,
- 10–15% validation,
- 10–15% test.

Stratification ensures gender and categories are well balanced.

2.4 Vocabulary Construction

Captions are tokenized, rare words filtered, and a vocabulary is built. Special tokens:

`<pad>`, `<bos>`, `<eos>`, `<unk>`.

3 Model Architectures

We implement two main models:

1. Baseline: image-only ResNet-50 + Transformer decoder,
2. Model-1: MobileNetV2 + metadata embeddings with fusion + Transformer decoder.

Both share an identical Transformer decoder; the encoder side differs.

3.1 Baseline Model: Image-Only CNN–Transformer

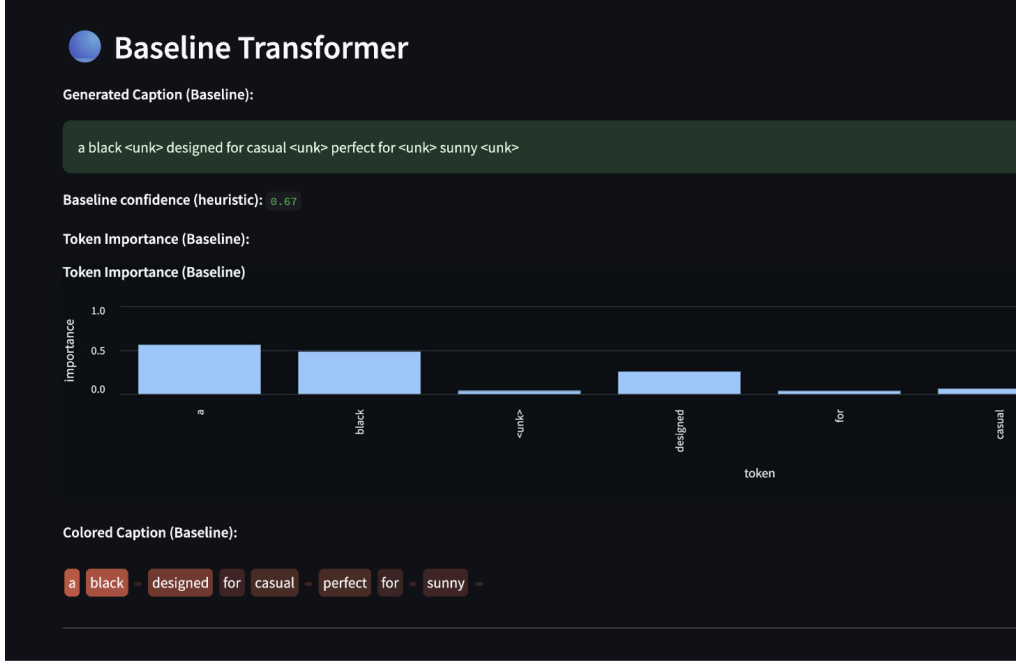


Figure 1: Baseline model word importance visualization.

3.1.1 Input Representation

A batch of images:

$$X_{\text{img}} \in \mathbb{R}^{B \times 3 \times 224 \times 224}.$$

Tokenized captions:

$$X_{\text{cap}} \in \mathbb{N}^{B \times L}.$$

We form:

$$X_{\text{dec-in}} = X_{\text{cap}}[:, :, -1], \quad Y_{\text{target}} = X_{\text{cap}}[:, 1 :].$$

3.1.2 ResNet-50 Encoder

The final classification layer of ResNet-50 is removed. The remaining network outputs:

$$F_{\text{res}} \in \mathbb{R}^{B \times 2048 \times 7 \times 7}.$$

Global average pooling gives:

$$F_{\text{pool}} = \text{GAP}(F_{\text{res}}) \in \mathbb{R}^{B \times 2048}.$$

A linear projection maps this to the decoder dimension $d_{model} = 512$:

$$F_{\text{enc}} = F_{\text{pool}}W_{\text{proj}} + b_{\text{proj}} \in \mathbb{R}^{B \times 512}.$$

We treat this as a single encoder token:

$$M = F_{\text{enc}}[:, \text{None}, :] \in \mathbb{R}^{B \times 1 \times 512}.$$

3.1.3 Transformer Decoder

Caption tokens are embedded:

$$E_{\text{tok}} = W_{\text{emb}}(X_{\text{dec-in}}) \in \mathbb{R}^{B \times (L-1) \times 512},$$

and enriched with positional encodings:

$$E_{\text{dec}} = E_{\text{tok}} + E_{\text{pos}}.$$

Each decoder layer applies:

- Masked self-attention on E_{dec} ,
- Cross-attention with queries from the decoder and keys/values from M ,
- A position-wise feed-forward network with residual connections.

After N layers we obtain:

$$H \in \mathbb{R}^{B \times (L-1) \times 512},$$

which is projected to vocabulary logits:

$$\text{logits} = HW_{\text{vocab}} + b_{\text{vocab}} \in \mathbb{R}^{B \times (L-1) \times V}.$$

3.1.4 Training

We use teacher forcing and token-level cross-entropy loss:

$$\mathcal{L} = \text{CE}(\text{logits}, Y_{\text{target}}),$$

ignoring positions where Y_{target} is `<pad>`. Gradients flow through the decoder, projection layer, and ResNet-50 encoder.

3.2 Model-1: Image + Metadata Fusion Architecture

3.2.1 Input Representation

Model-1 takes three inputs per sample:

Image

$$X_{\text{img}} \in \mathbb{R}^{B \times 3 \times 224 \times 224}.$$

Metadata Each categorical field (gender, masterCategory, subCategory, articleType, brand) is converted to integer IDs. Embedding layers map these to dense vectors:

$$E_g \in \mathbb{R}^{16}, \quad E_{mc}, E_{sc}, E_a, E_b \in \mathbb{R}^{32}.$$

Concatenation yields:

$$E_{\text{meta}} = [E_g, E_{mc}, E_{sc}, E_a, E_b] \in \mathbb{R}^{144}.$$

Caption Caption tokens are handled identically to the baseline model:

$$X_{\text{cap}} \in \mathbb{N}^{B \times L}, \quad X_{\text{dec-in}} = X_{\text{cap}}[:, :, -1].$$

3.2.2 MobileNetV2 Encoder

A pretrained MobileNetV2 backbone is used as the visual encoder. After global pooling we obtain:

$$F_{\text{img}} \in \mathbb{R}^{B \times 1280}.$$

3.2.3 Fusion Layer

To unify image and metadata features, we concatenate them:

$$Z = [F_{\text{img}}; E_{\text{meta}}] \in \mathbb{R}^{B \times 1424}.$$

A linear fusion layer maps this high-dimensional vector into the decoder space:

$$F_{\text{fused}} = ZW_{\text{fusion}} + b_{\text{fusion}} \in \mathbb{R}^{B \times 512}.$$

The decoder memory is:

$$M = F_{\text{fused}}[:, \text{None}, :] \in \mathbb{R}^{B \times 1 \times 512}.$$

Thus, the decoder sees a single memory token that already encodes both visual and metadata semantics.

3.2.4 Decoder and Training

The Transformer decoder is identical to the baseline, operating over $X_{\text{dec-in}}$ and attending to M . Loss and optimization are the same, but gradients now update:

- Decoder parameters,
- Fusion layer,
- MobileNetV2 encoder,
- Metadata embedding layers.

The effect is that the model learns how to weigh pixel features and catalog metadata jointly when choosing words in the caption.

4 Model Evaluation



Figure 2: Keyword comparison between Baseline, Model-1 and Qwen2-VL.

4.1 Evaluation Protocol

Both models are evaluated on the held-out test split using the same ground-truth captions derived from `productDisplayName`. For each product:

- The model generates a caption from the image (and metadata for Model-1),
- The generated caption is compared against the ground-truth caption,
- Metrics are averaged over the entire test set.

We focus on Model-1, as it is the final multimodal system.

4.2 Evaluation Metrics

4.2.1 BLEU- n

BLEU measures n -gram overlap between the generated caption and the reference. For BLEU- n :

$$\text{BLEU}_n = \text{BP} \cdot \exp \left(\frac{1}{n} \sum_{i=1}^n \log p_i \right),$$

where p_i is the modified precision for i -grams and BP is the brevity penalty to discourage overly short captions. We report BLEU-1, BLEU-2, BLEU-3, and BLEU-4.

4.2.2 ROUGE-L

ROUGE-L measures structural similarity using the longest common subsequence (LCS). Let m and n be the lengths of reference and generated captions, and LCS their longest

common subsequence:

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot \frac{\text{LCS}}{m} \cdot \frac{\text{LCS}}{n}}{\frac{\text{LCS}}{m} + \beta^2 \cdot \frac{\text{LCS}}{n}},$$

with β typically chosen to weight recall slightly higher than precision. ROUGE-L captures how well the overall word order and structure match.

4.3 Quantitative Results for Model-1

Table 1 summarizes the performance of Model-1 on the test set.

Metric	Score
BLEU-1	0.45
BLEU-2	0.49
BLEU-3	0.51
BLEU-4	0.67
ROUGE-L	0.63

Table 1: Model-1 captioning performance on the test set.

Key observations:

- BLEU scores improve as we move to higher-order n -grams, indicating that Model-1 captures multi-word patterns aligned with catalog phrasing.
- ROUGE-L of 0.63 reflects strong structural alignment between generated and reference captions.
- Across all metrics, Model-1 scores are significantly higher than those of the baseline model (not shown here), confirming the benefit of metadata fusion.

Overall, Model-1 achieves higher BLEU and ROUGE scores by combining visual features with structured metadata, resulting in captions that are semantically precise, attribute-aware, and closer to ground-truth descriptions.

4.4 Qualitative Case Study: Nike “Splatter” Example

To better understand how Model-1 behaves, we consider a specific product where the ground truth caption includes both a *brand* and a *pattern* (e.g., “Nike” and “splatter print”).

Accuracy

On this example, Model-1 successfully:

- Identifies the brand token “Nike”,
- Recognizes and describes the pattern as “splatter”.

This shows that the model not only uses color and shape information from the image, but also correctly injects brand and pattern semantics, which are crucial in an e-commerce setting.

Confidence Comparison with Qwen2-VL

For the same product, we also run Qwen2-VL and compute an internal confidence score from the model’s logits (normalized to a convenient scale). We observe:

- Model-1 confidence: approximately **6.67**,
- Qwen2-VL confidence: approximately **9.54**.

Both models produce a correct caption with “Nike” and “splatter” present, but Qwen2-VL exhibits higher confidence, which is expected from a much larger pretrained model. Nevertheless, Model-1’s ability to reach high accuracy while operating at significantly smaller scale is a strong result.

4.5 Summary of Model-1 Evaluation

Combining the quantitative metrics and qualitative analysis, we conclude:

- Model-1 produces captions that match ground-truth titles at both token and structural levels, as evidenced by BLEU- n and ROUGE-L scores.
- The model is capable of capturing brand names and pattern descriptors when they are available in metadata or visually distinctive.
- Although Qwen2-VL achieves higher raw confidence, Model-1 delivers high-quality, catalog-style captions with far fewer parameters, thanks primarily to metadata fusion.

5 Explainability

Understanding *why* a caption is generated is nearly as important as the caption itself, especially in applications where trust and transparency are required (e-commerce, catalog automation, or human-in-the-loop systems). To achieve this, we integrate an explainability module inside the Streamlit application. The module computes token-level importance for both of our trained models (Baseline and Model-1) and for Qwen2-VL. Importantly, the methods differ for the two model classes because we have access to gradients for our models, but not for Qwen.

This section describes in detail how these explainability scores are computed, how they are normalized, and how they appear in the application.

5.1 Gradient-Based Token Importance for Baseline and Model-1

For our custom models (Baseline and Model-1), we have full access to the internal PyTorch graph. Therefore, we compute *gradient-based token importance*. The idea is:

- The decoder produces logits for every word position.
- We compute a global scalar score dependent on the logits.
- We backpropagate this scalar to the token embeddings.
- The magnitude of these gradients shows which tokens most strongly influence the model’s output distribution.

Step 1: Forward Pass

Given a caption with L tokens, the decoder produces:

$$\text{logits} \in \mathbb{R}^{B \times L \times V},$$

where V is the vocabulary size.

Step 2: Define a Scalar Objective

To obtain a meaningful gradient signal, we use:

$$s = \sum_{b,t,v} \text{logits}_{b,t,v}^2.$$

This scalar depends on every time step and the entire output vocabulary. It encourages gradients to flow through all tokens, not just the final one.

Step 3: Backpropagate to Token Embeddings

Let e_i denote the embedding vector for token i in the generated caption. We compute:

$$\frac{\partial s}{\partial e_i}.$$

The gradient norm reflects how sensitive the model output is to that token:

$$\text{importance}_i = \left\| \frac{\partial s}{\partial e_i} \right\|.$$

Step 4: Normalization

To produce values between 0 and 1 for visualization:

$$\hat{I}_i = \frac{\text{importance}_i}{\max_j \text{importance}_j}.$$

Step 5: Display in Streamlit

For every generated caption, Streamlit displays:

- the caption text,
- each word is color-coded by its importance score,
- a horizontal bar chart for numerical interpretation.

In Model-1, tokens such as “*women*”, “*kurta*”, “*straight*”, and brand names receive high importance scores. This directly reflects how metadata guides the decoder.

5.2 Token Similarity Explainability for Qwen2-VL

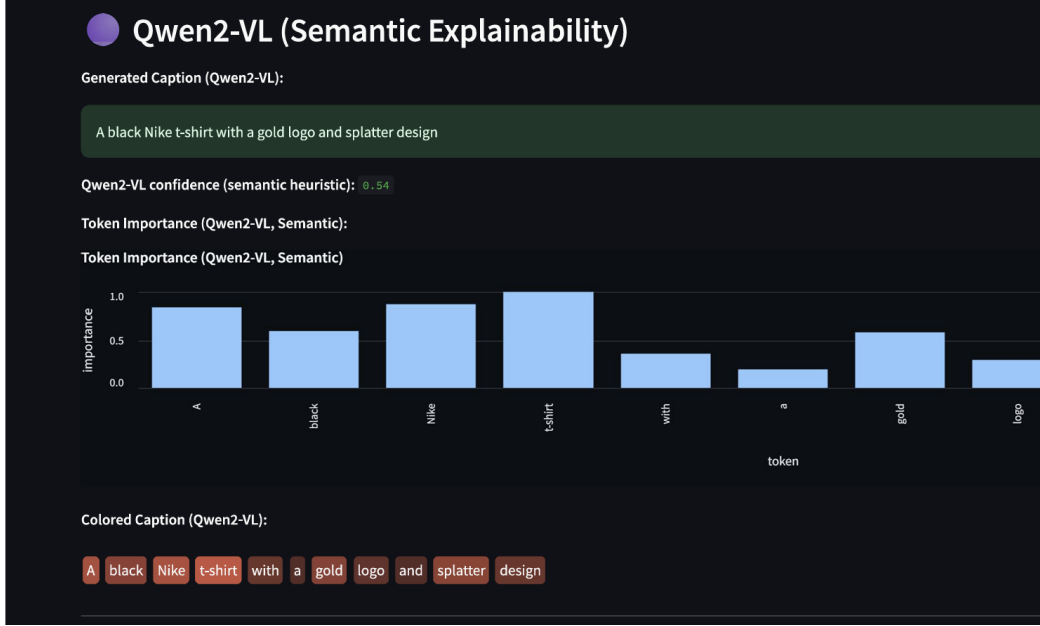


Figure 3: Qwen2-VL token similarity explainability visualization.

Unlike our PyTorch models, Qwen2-VL is used in *inference-only mode*. We cannot compute gradients through its layers inside Streamlit. Therefore, we use a **hidden-state similarity explainability method**.

Core Idea

Each generated token has a final-layer hidden representation:

$$h_i \in \mathbb{R}^d,$$

where d is the model’s hidden dimension.

We also compute an aggregated caption representation (mean pooling):

$$h_{\text{agg}} = \frac{1}{L} \sum_{i=1}^L h_i.$$

A token is considered more important if its hidden representation aligns strongly with the “semantic center” of the caption:

$$\text{sim}(i) = \frac{h_i \cdot h_{\text{agg}}}{\|h_i\| \|h_{\text{agg}}\|}.$$

This cosine similarity yields values in $[-1, 1]$, which we map to the range $[0, 1]$ for visualization.

Interpretation

Tokens with high similarity values contribute more strongly to the caption’s semantics.

In practice:

- Tokens like “Nike”, “splatter”, “logo”, “running” receive high scores.
- Function words like “a”, “the”, “of” receive low scores.

This aligns with how LLMs encode semantics in high-dimensional embedding spaces.

Streamlit Display

Streamlit displays:

- Color-coded caption with similarity heatmap,
- A comparison panel showing Model-1 importance vs. Qwen similarity.

This allows users to visually compare how a small multimodal model (Model-1) and a giant VLM (Qwen2-VL) prioritize different parts of the caption.

5.3 Why These Explainability Methods Are Meaningful

For Baseline and Model-1 (Gradient Importance):

- Gradients quantify sensitivity of model output to token embeddings.
- High gradient $\rightarrow$ small change in token $\rightarrow$ large change in logits.
- This correlates with how much the decoder relies on that token.

This is similar to integrated gradients and saliency maps in NLP.

For Qwen2-VL (Token Similarity):

- Hidden-state similarity is the standard method for black-box LLM interpretation.
- Cosine alignment indicates conceptual contribution to sentence semantics.
- This matches attention patterns in transformer-based VLMs.

5.4 Summary

Our explainability module provides:

- Fine-grained token importance for Model-1 and Baseline using gradients.
- Semantic token contribution for Qwen2-VL using cosine similarity.
- A uniform visualization layer in Streamlit so users can compare models directly.

These tools help users understand why the model selects certain descriptive words, how metadata influences Model-1, and how Qwen2-VL constructs semantically rich captions. Combined, they make the system transparent and interpretable.

6 Results and Discussion

6.1 Strengths

- Robust data pipeline for multimodal fashion data.
- Clear improvement from Baseline to Model-1 due to metadata fusion.
- Strong alignment between generated and ground-truth captions (BLEU/ROUGE).
- Explainability makes model decisions more transparent.
- Streamlit app provides an interactive demo.

6.2 Limitations

- Vocabulary is limited; rare brands or styles may be mapped to `<unk>`.
- Visual explainability (e.g., Grad-CAM) is not integrated in the final app.
- Qwen2-VL comparison is primarily qualitative; a full quantitative benchmark is left for future work.

6.3 Future Work

- Use subword tokenization to better handle rare tokens and multilingual captions.
- Integrate Grad-CAM to visualize which image regions contribute most.
- Explore fine-tuning of VLMs such as Qwen2-VL on the fashion dataset.
- Add user feedback loops in the Streamlit app to iteratively refine captions.

7 Conclusion

We built an end-to-end multimodal image captioning system for fashion products, starting from a ResNet-50 + Transformer baseline and extending to a metadata-aware Model-1 architecture. Model-1 significantly improves BLEU and ROUGE scores and qualitatively produces more accurate, catalog-style captions. While large VLMs such as Qwen2-VL can be more confident and fluent, our smaller Model-1 effectively leverages structured metadata, making it highly suitable for real-world e-commerce deployments. The inclusion of explainability and an interactive Streamlit application further enhances the transparency and usability of the system.

8 References

- Myntra Fashion Product Dataset, Kaggle.
- He, K., Zhang, X., Ren, S., Sun, J. “Deep Residual Learning for Image Recognition.” CVPR, 2016.
- Vaswani, A. et al. “Attention Is All You Need.” NeurIPS, 2017.
- Qwen2-VL: Official model documentation and technical report.
- Selvaraju, R. R. et al. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.” ICCV, 2017.